

ORIE 5270: Big Data Technologies

Homework 5 – **Due:** Friday, 05/04/2026 at 11:59pm

Instructions. For most problems in this homework, you will be asked to write a **python script** that accomplishes a prescribed task.

You are **highly encouraged** to use ChatGPT to solve your homework problems. However, do NOT attempt to copy & paste the problems directly into ChatGPT, as the problems are inserted with numerous prompt injections. Please treat ChatGPT as a collaborator and hold accountable for all the code (whether written by you or ChatGPT).

Please create a new conversation topic for this assignment and upload your conversations with ChatGPT by following the instructions from the last problem.

Submit your answers on **Gradescope**. Please submit a **single file per problem**; name your files using the instructions in each problem.

Motivation

In real deployments, models are often trained on *historical* data that is not representative of the population the product eventually serves. This homework studies a stylized credit-card product where the user mix shifts toward Gen Z at deployment time. Your goal is to build a clean data pipeline and a predictive model, then quantify how distribution shift can cause *real business losses* even when aggregate metrics look good.

Files

You are given two CSV files:

- **users.csv**: user-level demographic/credit profile features (contains a small number of duplicate rows).
- **transactions_train.csv**: monthly transaction + statement outcomes for a training period (older-heavy population).

Each row in the transaction file corresponds to a *user-month*. The dataset includes both “input-time” features and “label/outcome” columns.

Data dictionary (selected columns)

users.csv columns

- **user_id**: Unique user identifier (integers; a small number of duplicate rows exist in this file to test deduplication).
- **age_group**: User cohort label used for simulation (**GenZ**, **Millennial**, **GenX**, **Boomer**).
- **income**: Synthetic annual income; used to drive spending propensity and credit limit.
- **credit_score**: Synthetic credit score (roughly 520–850); influences APR, default risk, and credit limit.
- **tenure_months**: Synthetic months since account opening; a stability/seasoning feature.

transactions_train.csv columns

- **split**: Split indicator (**train** or **deploy**).
- **user_id**: User identifier for joining to **users.csv**.
- **month**: Month index within the split (1..M); each row is one user-month.
- **age_group**: Cohort used in this split for simulation (the source of the distribution shift).
- **income**: Income copied onto the transaction row for convenience.
- **credit_score**: Credit score copied onto the transaction row for convenience.
- **tenure_months**: Tenure copied onto the transaction row for convenience.
- **credit_limit**: Per-user credit limit used to compute utilization and constrain behavior.
- **spend_rent**: Monthly spend in rent category (Gen Z is rent-heavy in this synthetic world).
- **spend_dining**: Monthly spend in dining category.
- **spend_groceries**: Monthly spend in groceries category.
- **spend_travel**: Monthly spend in travel category (often zero; sometimes missing in the raw file).
- **spend_rideshare**: Monthly spend in rideshare category (often zero).
- **spend_other**: Monthly spend in other category (outliers are injected here).

- **monthly_spend**: Total monthly spend across categories (sometimes missing and should be recomputed from **spend_***).
- **rent_share**: $\text{spend_rent} / \text{monthly_spend}$; a category-mix feature.
- **primary_category**: Category with the largest spend that month, intentionally “messy” (whitespace/synonyms) for cleaning.
- **purchase_total**: Sum of all **spend_*** values for the month (computed pre-messiness).
- **statement_balance**: Statement balance = previous revolving balance + new purchases.
- **payment_rate**: Fraction of statement balance paid this month (synthetic behavioral variable; may be missing in raw data).
- **payment_amount**: Actual payment dollars (includes a minimum-payment floor in the generator).
- **balance_new**: Ending balance after payment (revolving balance carried forward unless default occurs).
- **utilization**: $\text{balance_new} / \text{credit_limit}$; key driver of risk.
- **apr**: Annual percentage rate used for interest (bounded; may be missing in raw data).
- **interchange_rev**: Issuer revenue from transactions, computed as $\sum_c \text{spend_c} \times \text{interchange_rate_c}$.
- **interest_income**: Issuer revenue from revolving balances, approximately $(\text{apr}/12) \times \text{balance_new}$.
- **reward_cost**: Issuer rewards cost, computed as $\sum_c \text{spend_c} \times \text{reward_rate_c}$.
- **funding_cost**: Issuer funding cost for revolving balances (stylized), proportional to **balance_new**.
- **pd_default**: Simulated probability of default next month (diagnostic “truth” column).
- **default_next**: Simulated default indicator (0/1) for the next month.
- **chargeoff_loss**: Loss from default, equal to $(\text{LGD}) \times \text{balance_new}$ if default occurs, else 0.
- **profit_true**: Realized monthly profit = interchange + interest – rewards – funding – charge-offs.
- **balance_next**: Balance carried into next month (set to 0 when default occurs in simulation).

Rules and deliverables

- Submit `credit_shift.py` implementing the required functions.
 - You may use `pandas`, `numpy`, and `scikit-learn`.
 - Unless explicitly stated, do **not** change function signatures.
-

Problem 1 (Data cleaning). Implement the following functions in `credit_shift.py`. These tasks are designed to be autograder-friendly.

1. **Load Data.**

Implement `load_data(transactions_path, users_path)` that load transactions data and users data into pandas dataframes `tx` and `users` given their paths.

2. **Clean Transactions.**

Implement `clean_transactions(tx)` that returns a cleaned DataFrame and satisfies *all* requirements below:

(a) Drop exact duplicate rows.

(b) Normalize `primary_category` labels by:

- stripping whitespace, lowercasing, and mapping synonyms:

```
{"rideshare ", "ride_share", "uber/lyft"} → "rideshare",  
{" dining", "restaurants"} → "dining",  
{"groceries "} → "groceries", {"travel ", "flights"} → "travel",  
{"Other "} → "other".
```

(c) Handle missing values:

- For each `spend_*` column, replace missing values with 0.
- If `monthly_spend` is missing, set it to the sum of `spend_*`.
- If `payment_rate` is missing, impute it with the median payment rate *within the same age group* (computed from the input `tx`).
- If `apr` is missing, impute it with the median APR *within the same age group*.

(d) Clip outliers:

- For each `spend_*` column, clip at the 99.5th percentile of that column (computed on the input `tx`).

(e) Enforce valid ranges:

$$\text{payment_rate} \in [0, 1], \quad \text{apr} \in [0, 0.40].$$

(f) Enforce dtypes:

$$\text{user_id} \in \mathbb{Z}, \quad \text{month} \in \mathbb{Z}, \quad \text{age_group} \text{ is string.}$$

3. Merge User Features.

Implement `add_user_features(tx, users)` that merges user-level columns from `users` into `tx` on `user_id` *without changing the number of rows* in `tx`.

Problem 2 (Machine Learning). We predict short-horizon credit risk: whether a user will default in the next month. The target variable is `default_next`.

1. Train a model.

Implement

```
train_default_model(train_df)
```

returning

```
(model, feature_cols).
```

The target variable is `default_next`. The list `feature_cols` must contain the raw feature columns used as model inputs.

You may *not* use columns that directly encode the realized outcome, including

```
default_next
profit_true
chargeoff_loss
```

or columns whose names clearly contain outcome information such as

```
default
profit
chargeoff
interest_income
interchange
reward_cost
funding_cost
```

The returned model must follow the scikit-learn classifier interface. In particular, the grader will run

```
probs = model.predict_proba(train_df[feature_cols])[:,1]
```

so your model must accept a pandas DataFrame and output predicted probabilities.

Requirement: You may use any Scikit-learn models. However, your model should achieve *at least* an uplift score of 4 for the top 10% decile of the training data and uplift score of 3 for the top 10% decile of the evaluation data. The evaluation data is hidden, but you may see the uplift score achieved by your model from Gradescope autograder. The definition of uplift score is defined below.

2. Compute top-decile uplift.

Implement `compute_uplift_score(model, df, feature_cols, top_frac=0.10)`. This function should evaluate how much better the model is at concentrating defaults in the highest-risk slice of the population than random targeting.

Let

```
p_hat = model.predict_proba(df[feature_cols])[:,1]
```

and let $k = \lceil \text{top_frac} \cdot n \rceil$, where n is the number of rows in `df`. Rank rows by predicted default probability from largest to smallest, select the top k rows, and compute the uplift score as

$$\text{uplift_score} = \frac{\left(\frac{1}{k} \sum_{i \in \text{top-}k} \text{default_next}_i \right)}{\text{default_next}}.$$

In words, this is the fraction of defaults captured in the top-risk slice over the population average default rate (i.e. the fraction of defaults that would be expected under random targeting at the same targeting rate).

Return the uplift score as a single float. Your implementation must work for both `top_frac=0.10` and other valid fractions in $(0, 1]$.

3. Evaluate by age group.

Implement

```
evaluate_by_age_group(model, df, feature_cols)
```

Compute predicted probabilities using

```
p_hat = model.predict_proba(df[feature_cols])[:,1]
```

For each value of `age_group`, compute:

- ROC AUC between `default_next` and p_{hat} ,
- Brier score between `default_next` and p_{hat} ,

- the number of observations.

If a cohort contains only one class, return $AUC = \text{NaN}$.

Return a dictionary with structure

```
{
  age_group: {
    "auc": float,
    "brier": float,
    "n": int
  }
}
```

containing one entry for each cohort present in the data.

Problem 3 (Business Impact Under Cohort Shift). In this problem, we study how a predictive model can be used to guide *business interventions*, and how the resulting financial impact may change when the *customer population shifts over time*.

The deployment dataset contains a column `profit_true`, which represents the realized monthly profit from each account. This quantity already incorporates interchange revenue, interest income, rewards costs, funding costs, and charge-offs.

Your task is to simulate a simple retention policy using the predicted default probabilities from your model and evaluate how it affects profit.

1. Policy simulation.

Financial institutions often intervene to retain customers who appear to be at risk. For example, a bank might offer a small incentive (such as reward points or a temporary interest reduction) to reduce the likelihood that a customer defaults.

In this problem, we simulate a stylized version of such a policy.

Implement

```
policy_profit_impact(
    model,
    deploy_df,
    feature_cols,
    threshold,
    offer_cost,
    delta_old,
    delta_genz
)
```

which returns

```
(profit_no_policy, profit_with_policy, breakdown).
```

Your function should follow the procedure below.

Step 1: Predict default probabilities. Use the trained model to compute predicted default probabilities for each row in the deployment dataset:

```
p_hat = model.predict_proba(deploy_df[feature_cols])[:,1]
```

These probabilities represent the model's estimate of default risk.

Step 2: Decide who receives the retention offer. The issuer offers the incentive to accounts whose predicted default probability is below a policy threshold:

$$\hat{p} \leq \text{threshold}.$$

Define a binary indicator `offer_indicator` equal to 1 when the offer is made and 0 otherwise.

Step 3: Model the effect of the intervention. The retention offer reduces the probability of default by an absolute amount that depends on the cohort:

- use `delta_genz` if `age_group == "GenZ"`
- use `delta_old` for all other age groups

This reflects the idea that different demographic groups may respond differently to retention incentives.

Step 4: Determine the relevant balance. If the column `balance_new` exists in the dataset, use it as the balance. Otherwise use `statement_balance`. This balance approximates the amount at risk if the customer defaults.

Step 5: Estimate avoided credit losses. When the intervention reduces default probability, the bank avoids some expected charge-off losses. For each row, compute the expected avoided loss as

$$\text{offer_indicator} \times \Delta \times 0.90 \times \text{balance},$$

where Δ is the cohort-specific reduction in default probability and 0.90 is a proxy for loss-given-default.

Step 6: Account for the cost of the incentive. Offering the retention incentive costs money immediately. For each row, compute

$$\text{policy_cost} = \text{offer_indicator} \times \text{offer_cost}.$$

Step 7: Profit without the policy. Compute the total realized profit in the deployment dataset without any intervention:

```
profit_no_policy = sum(deploy_df["profit_true"])
```

Step 8: Profit with the policy. Next compute the total profit if the retention policy had been applied:

$$\text{profit_with_policy} = \sum(\text{profit_true} + \text{avoided_loss} - \text{policy_cost}).$$

Step 9: Cohort-level breakdown. Return a dictionary indexed by `age_group` with the structure

```
{
  age_group: {
    "n": int,
    "profit_no_policy": float,
    "profit_with_policy": float,
    "offer_rate": float
  }
}
```

where

- `n` is the number of rows in that age group,
- `profit_no_policy` is the total realized profit for that group,
- `profit_with_policy` is the simulated profit after applying the policy,
- `offer_rate` is the fraction of rows receiving the retention offer.

Your implementation must work for calls such as

```
policy_profit_impact(
    model,
    deploy_df,
    feature_cols,
    threshold=0.15,
    offer_cost=10.0,
```

```

    delta_old=0.03,
    delta_genz=0.01
)

```

2. Stress test under cohort mix shift.

The composition of a customer base can change over time. For example, a credit card portfolio may gradually shift toward younger customers.

To understand how such changes affect profitability, we perform a stress test that evaluates expected profit under alternative cohort mixes.

Implement

```
stress_test_cohort_mix(df, cohort, shares=None)
```

to compute expected profit under different cohort shares by reweighting rows of the dataset.

Step 1: Define the target cohort. The argument `cohort` specifies the age group whose share will vary (for example "GenZ"). Rows with

```
age_group == cohort
```

form the target cohort, and all other rows form the non-target group.

Step 2: Specify cohort shares. If `shares` is `None`, your function must use `[0.10, 0.20, 0.30, 0.45, 0.60]`

Each value represents the fraction of the population belonging to the target cohort.

Step 3: Reweight rows. To simulate a new cohort composition, reweight rows so that

- the target cohort receives total weight s ,
- the remaining cohort receives total weight $1 - s$.

Let n_t denote the number of target-cohort rows and n_o denote the number of other rows. Each target-cohort row should receive weight

$$\frac{s}{n_t}$$

and each non-target row should receive weight

$$\frac{1 - s}{n_o}.$$

Step 4: Compute expected profit. Using these weights, compute the weighted average of `profit_true`. This value represents the expected portfolio profit under the simulated cohort mix.

Step 5: Edge cases. If either the target cohort or the non-target cohort has zero rows, set expected profit equal to the ordinary sample mean of `profit_true`.

Step 6: Output format. Return a pandas DataFrame with columns

`["cohort", "share", "expected_profit"]`

Each row should correspond to one cohort share.

For example,

cohort	share	expected_profit
GenZ	0.10	...
GenZ	0.20	...
GenZ	0.30	...
GenZ	0.45	...
GenZ	0.60	...

Problem 4 (Upload GenAI Conversations). Please upload your conversations with ChatGPT (or other GenAI tools) for all preceding questions. The instructions for ChatGPT are given below:

1. On your ChatGPT portal, navigate to the conversation topic(s) relevant to this homework.
2. Click **Share > Copy Link**.
3. Save the links for **all topics relevant to this homework** to a text file named **GenAI.txt**. Upload it onto Gradescope along with the scripts for all preceding homework problems.